Teste de Mutação com StrykerJS

Avaliação e fortalecimento da suíte de testes do projeto *operacoes-mutante*

Autoria luisajardim

Repositório https://github.com/luisajardim/operacoes-mutante-PUC

Data 12 de outubro de 2026

Indicador	Inicial	Final
Testes	50	85
Cobertura de linhas	98,64%	100%
Cobertura de branches	58,82%	100%
Mutation score	**73,71%**	**97,65%**

1. Introdução

O projeto analisado é uma biblioteca JavaScript com 50 funções de cálculo (`src/operacoes.js`) e uma suíte de 50 testes em Jest. A suíte foi construída para ter **cobertura de código alta**, mas com asserções fracas. O objetivo deste trabalho foi usar **teste de mutação** para medir a qualidade real dessa suíte, identificar os mutantes que sobreviveram e escrever testes que os matem, sem alterar o código de produção.

No teste de mutação, a ferramenta gera versões levemente alteradas do código (os *mutantes*), por exemplo trocando `<` por `<=` ou removendo uma chamada de método, e roda a suíte contra cada uma. Se algum teste falha, o mutante é considerado **morto**. Se todos passam, o mutante **sobreviveu**, o que indica uma mudança de comportamento que os testes não detectam.

2. Ambiente e configuração

- Node.js 22.17.0, Jest 29.7, StrykerJS 10.0.0 com `@stryker-mutator/jest-runner`.
- O `npx stryker init` é interativo, então foi criado manualmente um `stryker.conf.json` equivalente:

```
{
  "packageManager": "npm",
  "testRunner": "jest",
  "mutate": ["src/**/*.js"],
  "reporters": ["html", "clear-text", "progress", "json"],
  "coverageAnalysis": "perTest"
}
```

- As pastas `reports/` e `.stryker-tmp/` foram adicionadas ao `.gitignore`.
- Restrição seguida durante todo o trabalho: **nenhuma alteração em `src/`**. Todo teste novo passa com o código original.

3. Análise inicial

3.1 Cobertura de código

Statements	Branches	Functions	Lines
85,41%	58,82%	100%	98,64%

A única linha não executada era a 112 (ramo de array com quantidade par em `medianaArray`).

3.2 Mutation score

Total de mutantes	Killed	Timeout	Survived	No coverage	Mutation score
213	154	3	44	12	**73,71%** (78,11% sobre o código coberto)

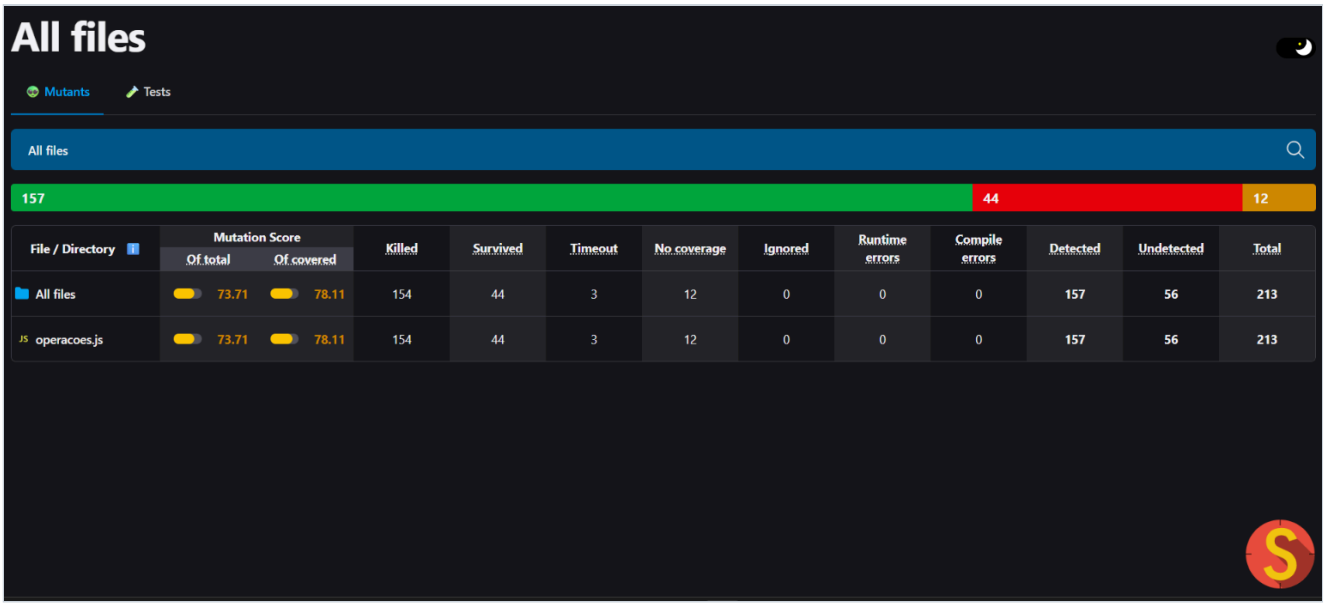


Figura 1: resumo da primeira execução. Em vermelho, os 44 sobreviventes; em laranja, os 12 sem cobertura.



Figura 2: código filtrado para os mutantes não detectados. Cada bolinha no fim da linha é um mutante (vermelho = sobreviveu, laranja = sem cobertura). Na linha 8 aparece o mutante da mensagem de erro de `divisao`; na linha 13, a fronteira `n < 0` de `raizQuadrada`.

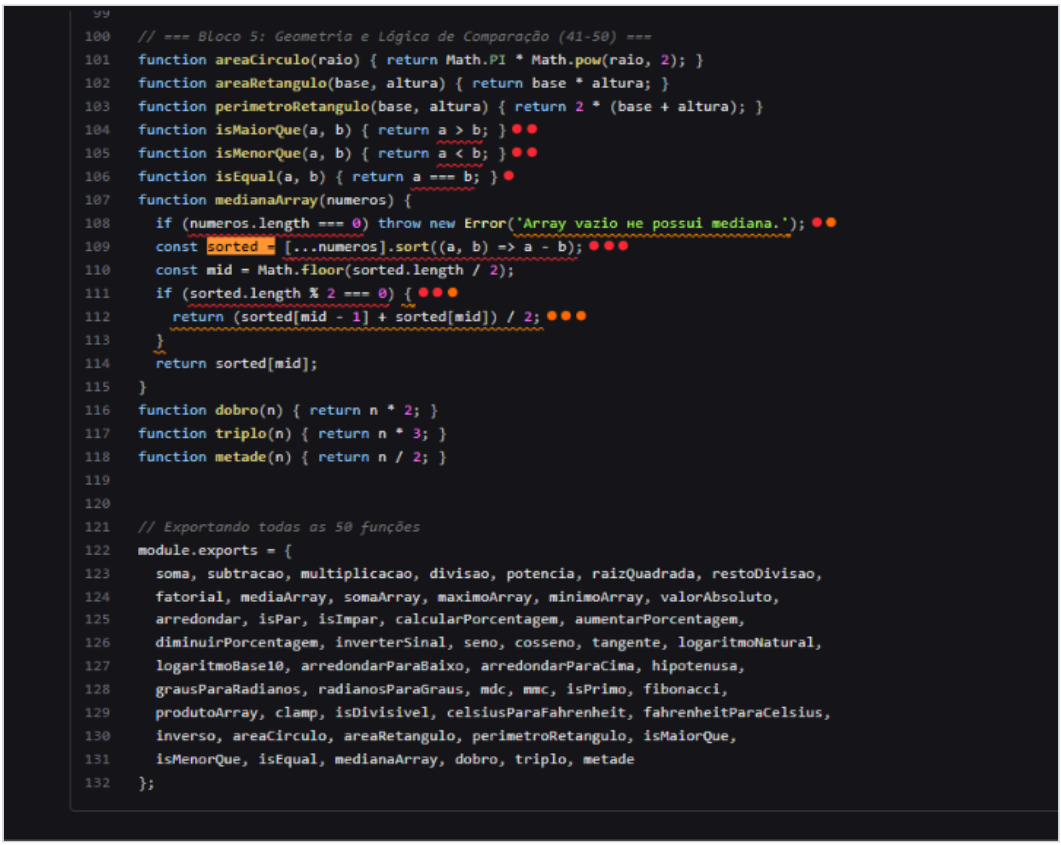
3.3 Por que cobertura e mutation score são tão diferentes?

A cobertura mede apenas se uma linha **foi executada** durante os testes. Ela não verifica se o teste **conferiu** o resultado. O teste de mutação mede outra coisa: se a suíte **percebe** quando o código muda. Com 98,64% das linhas cobertas, mais de um quarto dos mutantes (56 de 213) passou despercebido, porque a suíte original:

- **Testava só o "caminho feliz" dos predicados.** `isPar`, `isImpar`, `isEqual`, `isMaiorQue`, `isMenorQue` e `isDivisivel` só eram testados com entradas que devolvem `true`. Por isso um mutante que troca a expressão por `return true` passava.
- **Não testava as fronteiras.** Não havia testes com `a === b`, `n = 0` ou `n = 1`. Por isso as trocas `<` → `<=` e `>` → `>=` sobreviviam.

- **Usava `toThrow()` sem mensagem.** Um mutante que troca a mensagem por `""` continuava lançando erro e passava no teste.
- **Executava guardas sem exercitar o ramo.** A linha `if (numeros.length === 0) ...` conta como coberta mesmo que nenhum teste use um array vazio, então o mutante `if (false)` sobrevivia.
- **Escolhia entradas que escondem o erro.** `medianaArray` era testada com um array já ordenado, então remover o `sort` não mudava nada. As conversões de temperatura usavam 0 °C ↔ 32 °F, valores em que `9/5` e `5/9` são multiplicados por zero, então trocar `*` por `/` dava o mesmo resultado. `isPrimo` só era testada com o primo 7, então apagar o corpo do laço (que é o que detecta compostos) não fazia diferença.

A cobertura de **branches** (58,82%) já indicava o problema: muitos `if` nunca tinham o ramo verdadeiro executado. Cobertura alta é condição necessária, mas não suficiente.



```
100  // === Bloco 5: Geometria e Lógica de Comparação (41-50) ===
101  function areaCirculo(raio) { return Math.PI * Math.pow(raio, 2); }
102  function areaRetangulo(base, altura) { return base * altura; }
103  function perimetroRetangulo(base, altura) { return 2 * (base + altura); }
104  function isMaiorQue(a, b) { return a > b; }
105  function isMenorQue(a, b) { return a < b; }
106  function isEqual(a, b) { return a === b; }
107  function medianaArray(numeros) {
108    if (numeros.length === 0) throw new Error('Array vazio não possui mediana.');
109    const sorted = [...numeros].sort((a, b) => a - b);
110    const mid = Math.floor(sorted.length / 2);
111    if (sorted.length % 2 === 0) {
112      return (sorted[mid - 1] + sorted[mid]) / 2;
113    }
114    return sorted[mid];
115  }
116  function dobro(n) { return n * 2; }
117  function triplo(n) { return n * 3; }
118  function metade(n) { return n / 2; }
119
120
121  // Exportando todas as 50 funções
122  module.exports = {
123    soma, subtracao, multiplicacao, divisao, potencia, raizQuadrada, restoDivisao,
124    fatorial, mediaArray, somaArray, maximoArray, minimoArray, valorAbsoluto,
125    arredondar, isPar, isImpar, calcularPorcentagem, aumentarPorcentagem,
126    diminuirPorcentagem, inverterSinal, seno, cosseno, tangente, logaritmoNatural,
127    logaritmoBase10, arredondarParaBaixo, arredondarParaCima, hipotenusa,
128    grausParaRadianos, radianosParaGraus, mdc, mmc, isPrimo, fibonacci,
129    produtoArray, clamp, isDivisivel, celsiusParaFahrenheit, fahrenheitParaCelsius,
130    inverso, areaCirculo, areaRetangulo, perimetroRetangulo, isMaiorQue,
131    isMenorQue, isEqual, medianaArray, dobro, triplo, metade
132  };
```

Figura 3: bloco 5 do código. Os predicados `isMaiorQue`, `isMenorQue` e `isEqual` (linhas 104–106) e quase toda a `medianaArray` têm sobreviventes. A linha 112 (bolinhas laranja) é a única que a cobertura acusava como não executada; todas as outras bolinhas vermelhas estão em linhas cobertas.

4. Os três mutantes sobreviventes mais interessantes

Os números entre `#` referem-se à tabela completa do Apêndice A.

4.1 `medianaArray`: o `sort` inteiro pode ser apagado (linha 109)

```
-   const sorted = [...numeros].sort((a, b) => a - b);
+   const sorted = [...numeros];
```

Mutador: **MethodExpression** (#48). Na mesma linha também sobreviveram `sort(() => undefined)` (#49, ArrowFunction) e `a + b` no comparador (#50, ArithmeticOperator).

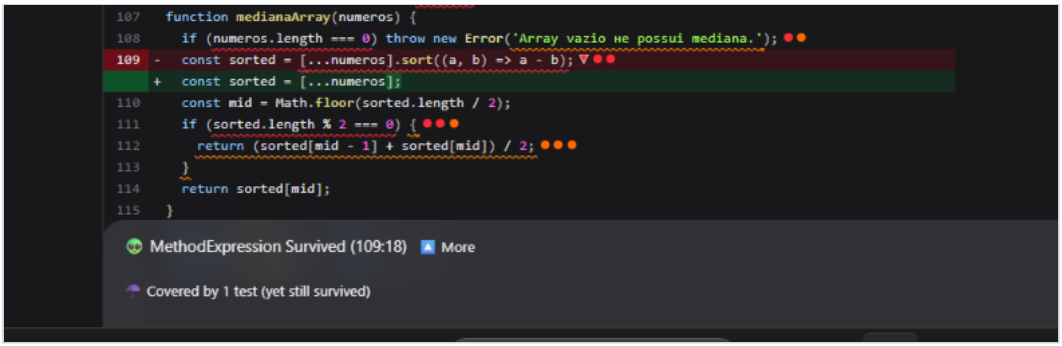


Figura 4: o Stryker indica "Covered by 1 test (yet still survived)". A linha foi executada por um teste, mas o teste não percebeu a remoção do `sort`.

Por que sobreviveu: o único teste usava `[1, 2, 3, 4, 5]`, que **já está ordenado**. Com ou sem ordenação, o elemento central é 3. A linha foi executada (por isso estava coberta), mas a ordenação nunca fez diferença no resultado.

Teste que o mata:

```
test('ordena antes de escolher o elemento central (array ímpar desordenado) [48-50]', () => {
  expect(medianaArray([5, 1, 3])).toBe(3);
});
```

Sem o `sort`, o elemento central de `[5, 1, 3]` é 1, e o teste falha. O comparador `a + b` é sempre positivo para esses números, então o V8 deixa o array na ordem original e o teste também mata esse mutante.

4.2 `isPrimo`: o laço que detecta compostos pode ser desativado (linha 74)

```
-   for (let i = 2; i < n; i++) {
-     if (n % i === 0) return false;
-   }
+   for (let i = 2; i < n; i++) {}
```

Mutador: **BlockStatement** (#25). Na mesma linha sobreviveram também `i < n` → `false` (#23) e `i < n` → `i >= n` (#24), além de `n % i === 0` → `false` (#26) e `n % i` → `n * i` (#27) na linha seguinte. O `return false` (#22, #28) e a guarda `n <= 1` (#20, #21) também não tinham cobertura ou sobreviviam.

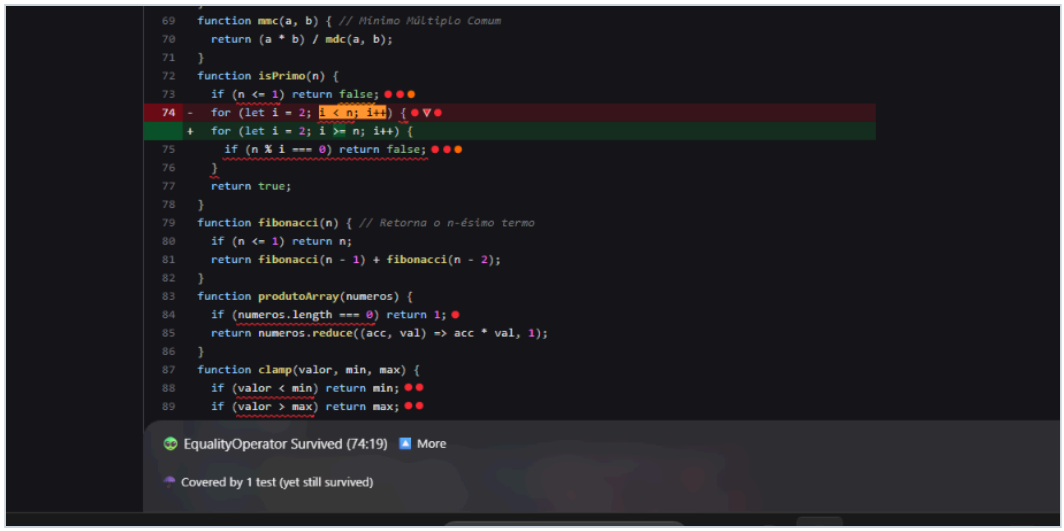


Figura 5: mutação `i < n` → `i >= n` (#24), da mesma linha. O laço nunca executa e a função devolve `true` para todo `n > 1`, e mesmo assim o teste com `isPrimo(7)` passa. A imagem mostra também o mutante equivalente de `produtoArray` (linha 84) e os sobreviventes de `clamp` (linhas 88–89).

Por que sobreviveu: o teste só verificava `isPrimo(7) === true`. Uma função que devolve `true` para qualquer `n > 1` passa nesse teste. A lógica que identifica números compostos nunca foi exercitada.

Testes que o matam:

```
test('números compostos não são primos [23-28]', () => {
  expect(isPrimo(4)).toBe(false);
  expect(isPrimo(9)).toBe(false);
  expect(isPrimo(15)).toBe(false);
});

test('1 não é primo [20, 21, 22]', () => {
  expect(isPrimo(1)).toBe(false);
});
```

4.3 `clamp`: `<` → `<=`, que só é detectável com `-0` (linha 88)

```
-   if (valor < min) return min;
+   if (valor <= min) return min;
```

Mutador: **EqualityOperator** (#31). A versão simétrica, `valor > max` → `valor >= max`, também sobreviveu (#33).

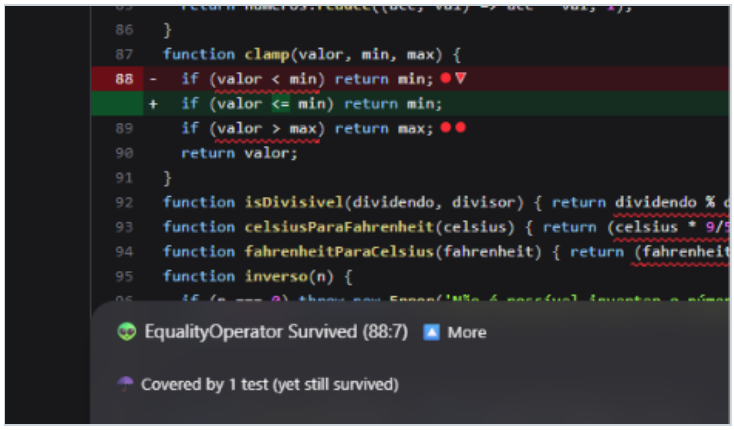


Figura 6: `valor < min` trocado por `valor <= min`, sobrevivente na primeira execução.

Por que sobreviveu: o teste original só usava `clamp(5, 0, 10)`, um valor no meio do intervalo. Além disso, este mutante é **quase equivalente**. Quando `valor === min`, a função original devolve `valor` e a mutante devolve `min`, que são o mesmo número. Por isso o teste de fronteira óbvio, `clamp(0, 0, 10)`, **não** mata o mutante.

Testes que o matam: o único caso em que `valor === min` vale e os dois números ainda são distinguíveis é o par `0` / `-0`. O `toBe` do Jest usa `Object.is`, que diferencia os dois:

```
test('retorna o próprio valor (não o limite) quando valor === min [31]', () => {
  // -0 e 0 são iguais para <, mas distintos para Object.is (usado pelo toBe):
  // com "valor <= min" a função devolveria min (0) em vez do próprio valor (-0).
  expect(clamp(-0, 0, 10)).toBe(-0);
});

test('retorna o próprio valor (não o limite) quando valor === max [33]', () => {
  expect(clamp(0, -10, -0)).toBe(0);
});
```

É um caso limite. A diferença é real e observável por quem chama a função (por exemplo, `1 / clamp(-0, 0, 10)` dá `-Infinity` no original e `Infinity` na mutante), mas dificilmente importaria na prática. O caso mostra como é tênue a linha entre um mutante "difícil" e um equivalente.

5. Melhorias na suíte de testes

Foram escritos **35 testes novos** no arquivo `test/operacoes.mutantes.test.js`, sem alterar os 50 originais. Eles estão agrupados por comportamento verificado:

Grupo (`describe`)	O que verifica	Mutantes mortos
Mensagens de erro e guardas de entrada inválida	`toThrow` com a mensagem exata em `divisao`, `raizQuadrada`, `fatorial`, `maximoArray`, `minimoArray`, `inverso`, `medianaArray`	13
Valores na fronteira das guardas	`raizQuadrada(0)`, `fatorial(0)` e `fatorial(1)` não lançam erro	2

Grupo (`describe`)	O que verifica	Mutantes mortos
Arrays vazios com valor de retorno definido	`mediaArray([])` → 0 e `produtoArray([])` → 1	1
Predicados retornam `false`	`isPar`, `isImpar`, `isDivisivel` e `isEqual` com entradas que devolvem `false`	5
Comparações estritas	`isMaiorQue` e `isMenorQue` com `a === b` e com a ordem invertida	4
`isPrimo`	0, 1, negativos, 2 e números compostos	9
`clamp`	valores abaixo, acima e exatamente nos limites	4
Conversões de temperatura	valores diferentes de zero (100 °C ↔ 212 °F, −40)	4
`medianaArray`	arrays desordenados, de tamanho par e preservação da entrada	9

Cada teste traz no nome os números (`[n]`) dos mutantes que mata, o que permite rastrear cada teste até o Apêndice A.

6. Resultados finais

6.1 Mutation score

Total de mutantes	Killed	Timeout	Survived	No coverage	Mutation score
213	205	3	5	0	**97,65%**

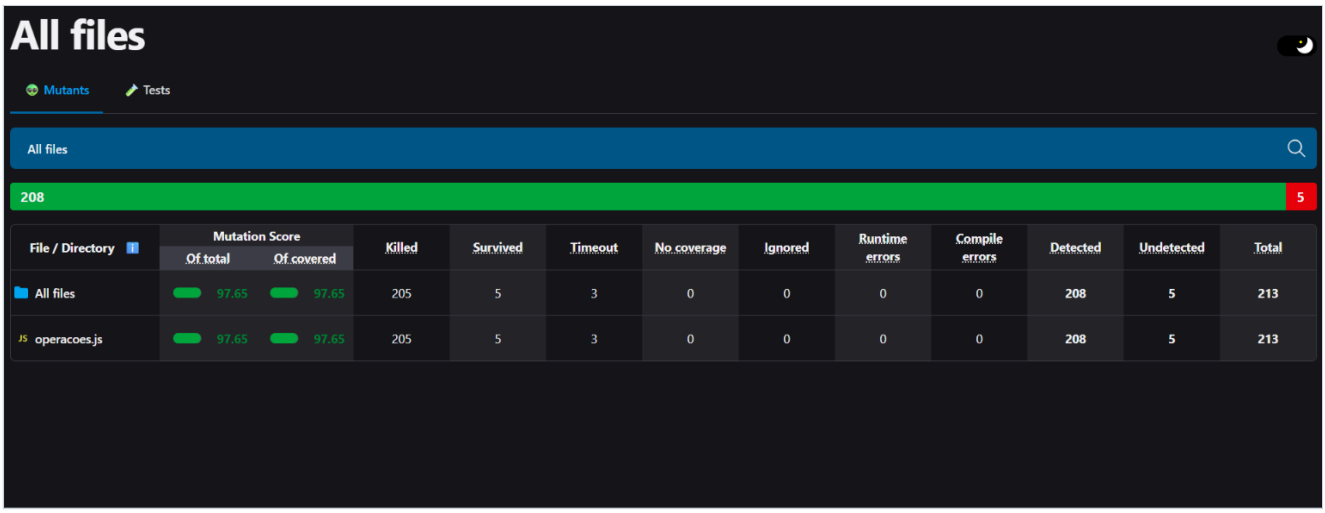


Figura 7: resumo da execução final. Restam apenas os 5 mutantes equivalentes, e não há mais mutantes sem cobertura.

Descontando os 5 mutantes equivalentes, o score sobre os mutantes que **podem** ser mortos é **100%** (208/208).

6.2 Cobertura de código

Statements	Branches	Functions	Lines
100%	100%	100%	100%

6.3 Sobre a meta de 98% e os mutantes equivalentes

Os 5 sobreviventes restantes são **mutantes equivalentes**: o comportamento observável (o valor de retorno para qualquer entrada) é idêntico ao do código original. Com 213 mutantes, o máximo alcançável sem testes artificiais é 208/213 = **97,65%**, ligeiramente abaixo dos 98%. Para passar dessa marca seria preciso, por exemplo, espionar `Array.prototype.reduce` e verificar se ele foi chamado em `produtoArray([])`. Isso testaria um detalhe de implementação, não o comportamento, e por isso foi deliberadamente evitado.

Equivalentes 1 a 4: `fatorial`, linha 19 (#8, #9, #10, #11)

```
if (n === 0 || n === 1) return 1;     // original
if (false) return 1;                  // #9
if (n === 0 && n === 1) return 1;     // #8 (nunca verdadeiro, igual a "false")
if (false || n === 1) return 1;       // #10
if (n === 0 || false) return 1;       // #11
```

Esse `if` é apenas um atalho. Sem ele, para `n = 0` ou `n = 1`, a execução segue para `let resultado = 1; for (let i = 2; i <= n; i++)`. Como `2 <= 0` e `2 <= 1` são falsos, o laço não executa e a função devolve `resultado = 1`, exatamente o mesmo valor. Nenhuma entrada distingue as versões. O teste `fatorial(0) === 1` existe, mas não consegue matar esses mutantes.

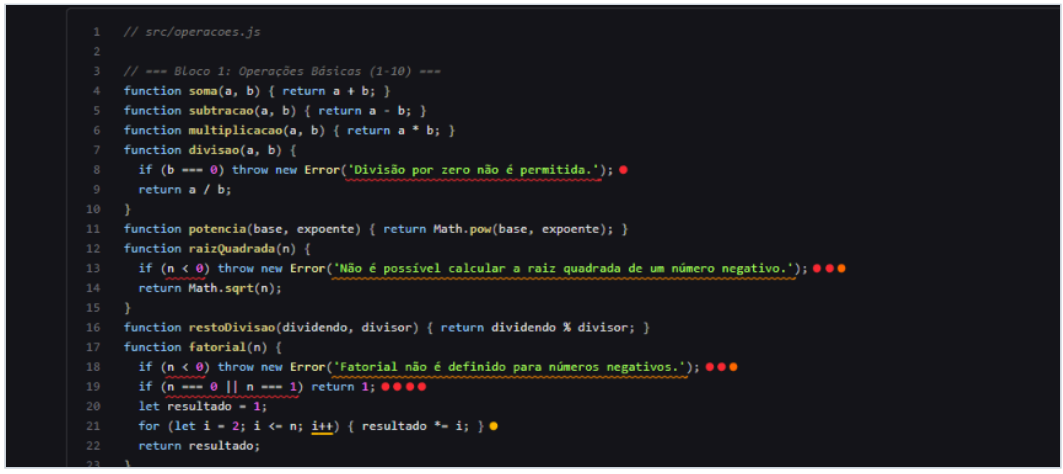


Figura 8: as 4 bolinhas na linha 19 são os mutantes equivalentes de `fatorial`. A linha 21 tem um mutante em timeout (amarelo), que conta como detectado.

Equivalente 5: `produtoArray`, linha 84 (#29)

```
if (numeros.length === 0) return 1;   // original
if (false) return 1;                  // mutante
```

Sem a guarda, `[].reduce((acc, val) => acc * val, 1)` devolve o valor inicial, `1`. O resultado é idêntico.

7. Conclusão

A suíte original tinha 98,64% de cobertura de linhas e 100% de funções, mas deixava passar mais de um quarto dos mutantes. Os testes executavam quase todo o código, mas pouco verificavam: asserções só no caminho feliz, nenhuma fronteira, mensagens de erro ignoradas e entradas que mascaravam a lógica. O teste de mutação expôs exatamente esses pontos, e cada sobrevivente apontou um teste concreto a escrever.

Com 35 testes direcionados, o mutation score subiu de **73,71% para 97,65%**, e a cobertura chegou a 100% em todas as métricas. Todos os mutantes que alteram o comportamento observável foram mortos. Os 5 restantes são equivalentes e foram justificados em vez de "mortos à força". O trabalho também mostrou que a cobertura é um indicador de onde **não** há testes, e não de quão bons eles são.

Apêndice A: mutantes não detectados na 1ª execução e como cada um foi resolvido

Resumo: **51 mortos** por novos testes e **5 equivalentes**.

#	Status inicial	Linha:Col	Mutador	Original	Mutado	Resolução
1	Survived	8:32	StringLiteral	`'Divisão por zero não é permitida.'`	`""`	Morto — `toThrow` com a mensagem exata
2	Survived	13:7	ConditionalExpression	`n < 0`	`false`	Morto — `raizQuadrada(-4)` deve lançar erro com mensagem
3	Survived	13:7	EqualityOperator	`n < 0`	`n <= 0`	Morto — fronteira `raizQuadrada(0)` → 0
4	NoCoverage	13:30	StringLiteral	`'Não é possível calcular a raiz quadrada de um número negativo.'`	`""`	Morto — mensagem verificada em `raizQuadrada(-4)`
5	Survived	18:7	ConditionalExpression	`n < 0`	`false`	Morto — `fatorial(-1)` deve lançar erro com mensagem
6	Survived	18:7	EqualityOperator	`n < 0`	`n <= 0`	Morto — fronteira `fatorial(0)` → 1
7	NoCoverage	18:30	StringLiteral	`'Fatorial não é definido para números negativos.'`	`""`	Morto — mensagem verificada em `fatorial(-1)`
8	Survived	19:7	LogicalOperator	`n === 0 || n === 1`	`n === 0 && n === 1`	**Equivalente** — para n=0 ou 1 o laço não executa e o retorno é 1 de qualquer forma
9	Survived	19:7	ConditionalExpression	`n === 0 || n === 1`	`false`	**Equivalente** — idem #8
10	Survived	19:7	ConditionalExpression	`n === 0`	`false`	**Equivalente** — idem #8
11	Survived	19:18	ConditionalExpression	`n === 1`	`false`	**Equivalente** — idem #8
12	Survived	25:7	ConditionalExpression	`numeros.length === 0`	`false`	Morto — `mediaArray([])` → 0 (mutante retorna NaN)
13	Survived	34:7	ConditionalExpression	`numeros.length === 0`	`false`	Morto — `maximoArray([])` deve lançar erro (mutante retorna -Infinity)
14	NoCoverage	34:45	StringLiteral	`'Array vazio не possui valor máximo.'`	`""`	Morto — mensagem exata verificada
15	Survived	38:7	ConditionalExpression	`numeros.length === 0`	`false`	Morto — `minimoArray([])` deve lançar erro (mutante retorna Infinity)
16	NoCoverage	38:45	StringLiteral	`'Array vazio не possui valor mínimo.'`	`""`	Morto — mensagem exata verificada
17	Survived	43:28	ConditionalExpression	`n % 2 === 0`	`true`	Morto — `isPar(7)` → false
18	Survived	44:30	ConditionalExpression	`n % 2 !== 0`	`true`	Morto — `isImpar(4)` → false
19	Survived	44:30	ArithmeticOperator	`n % 2`	`n * 2`	Morto — `isImpar(4)` → false (4*2 !== 0)

#	Status inicial	Linha:Col	Mutador	Original	Mutado	Resolução
20	Survived	73:7	ConditionalExpression	`n <= 1`	`false`	Morto — `isPrimo(1)` → false
21	Survived	73:7	EqualityOperator	`n <= 1`	`n < 1`	Morto — `isPrimo(1)` → false
22	NoCoverage	73:22	BooleanLiteral	`false`	`true`	Morto — `isPrimo(1)` → false
23	Survived	74:19	ConditionalExpression	`i < n`	`false`	Morto — compostos (`isPrimo(4)`, 9, 15) → false
24	Survived	74:19	EqualityOperator	`i < n`	`i >= n`	Morto — compostos → false
25	Survived	74:31	BlockStatement	`for (let i = 2; i < n; i++) { if (n % i === 0) return false; }`	`{}`	Morto — compostos → false
26	Survived	75:9	ConditionalExpression	`n % i === 0`	`false`	Morto — compostos → false
27	Survived	75:9	ArithmeticOperator	`n % i`	`n * i`	Morto — compostos → false
28	NoCoverage	75:29	BooleanLiteral	`false`	`true`	Morto — compostos → false
29	Survived	84:7	ConditionalExpression	`numeros.length === 0`	`false`	**Equivalente** — `[].reduce(fn, 1)` já retorna 1
30	Survived	88:7	ConditionalExpression	`valor < min`	`false`	Morto — `clamp(-5, 0, 10)` → 0
31	Survived	88:7	EqualityOperator	`valor < min`	`valor <= min`	Morto — `clamp(-0, 0, 10)` deve devolver -0 (o próprio valor), não o limite 0
32	Survived	89:7	ConditionalExpression	`valor > max`	`false`	Morto — `clamp(15, 0, 10)` → 10
33	Survived	89:7	EqualityOperator	`valor > max`	`valor >= max`	Morto — `clamp(0, -10, -0)` deve devolver 0, não o limite -0
34	Survived	92:51	ConditionalExpression	`dividendo % divisor === 0`	`true`	Morto — `isDivisivel(10, 3)` → false
35	Survived	93:51	ArithmeticOperator	`celsius * 9/5`	`celsius * 9 * 5`	Morto — `celsiusParaFahrenheit(100)` → 212
36	Survived	93:51	ArithmeticOperator	`celsius * 9`	`celsius / 9`	Morto — `celsiusParaFahrenheit(100)` → 212
37	Survived	94:53	ArithmeticOperator	`(fahrenheit - 32) * 5/9`	`(fahrenheit - 32) * 5 * 9`	Morto — `fahrenheitParaCelsius(212)` → 100
38	Survived	94:53	ArithmeticOperator	`(fahrenheit - 32) * 5`	`(fahrenheit - 32) / 5`	Morto — `fahrenheitParaCelsius(212)` → 100
39	Survived	96:7	ConditionalExpression	`n === 0`	`false`	Morto — `inverso(0)` deve lançar erro (mutante retorna Infinity)
40	NoCoverage	96:32	StringLiteral	`'Não é possível inverter o número zero.'`	`""`	Morto — mensagem exata verificada

#	Status inicial	Linha:Col	Mutador	Original	Mutado	Resolução
41	Survived	104:36	ConditionalExpression	`a > b`	`true`	Morto — `isMaiorQue(5, 10)` → false
42	Survived	104:36	EqualityOperator	`a > b`	`a >= b`	Morto — fronteira `isMaiorQue(5, 5)` → false
43	Survived	105:36	ConditionalExpression	`a < b`	`true`	Morto — `isMenorQue(10, 5)` → false
44	Survived	105:36	EqualityOperator	`a < b`	`a <= b`	Morto — fronteira `isMenorQue(5, 5)` → false
45	Survived	106:33	ConditionalExpression	`a === b`	`true`	Morto — `isEqual(7, 8)` → false
46	Survived	108:7	ConditionalExpression	`numeros.length === 0`	`false`	Morto — `medianaArray([])` deve lançar erro
47	NoCoverage	108:45	StringLiteral	`'Array vazio не possui mediana.'`	`""`	Morto — mensagem exata verificada
48	Survived	109:18	MethodExpression	`[...numeros].sort((a, b) => a - b)`	`[...numeros]`	Morto — array desordenado `[5, 1, 3]` → 3
49	Survived	109:36	ArrowFunction	`(a, b) => a - b`	`() => undefined`	Morto — array desordenado `[5, 1, 3]` → 3
50	Survived	109:46	ArithmeticOperator	`a - b`	`a + b`	Morto — array desordenado `[5, 1, 3]` → 3
51	Survived	111:7	ConditionalExpression	`sorted.length % 2 === 0`	`false`	Morto — array par `[1, 2, 3, 4]` → 2.5
52	Survived	111:7	ArithmeticOperator	`sorted.length % 2`	`sorted.length * 2`	Morto — array par `[1, 2, 3, 4]` → 2.5
53	NoCoverage	111:32	BlockStatement	`if (sorted.length % 2 === 0) { return (sorted[mid - 1] + sorted[mid]) / 2; }`	`{}`	Morto — array par `[1, 2, 3, 4]` → 2.5
54	NoCoverage	112:12	ArithmeticOperator	`(sorted[mid - 1] + sorted[mid]) / 2`	`(sorted[mid - 1] + sorted[mid]) * 2`	Morto — array par `[1, 2, 3, 4]` → 2.5
55	NoCoverage	112:13	ArithmeticOperator	`sorted[mid - 1] + sorted[mid]`	`sorted[mid - 1] - sorted[mid]`	Morto — array par `[1, 2, 3, 4]` → 2.5
56	NoCoverage	112:20	ArithmeticOperator	`mid - 1`	`mid + 1`	Morto — array par `[1, 2, 3, 4]` → 2.5